

VQL LSP Server — Test Summary

This document summarizes every test performed on the VQL language server (velociraptor lsp) across the prototype milestones: the participle v2 migration, the Inspect() and Outline() APIs, the diagnostic engine, the artifact registry, the opencode integration, and document symbols.

Go unit test total: 30 tests in vql/lsp/ (10 diagnostic engine + 6 server lifecycle + 2 document symbols + 5 hover + 6 completion + 1 API-client artifact registry), plus 2 vfilter Inspect() tests, 2 vfilter Outline() tests, and the full vfilter migration suite.

1. vfilter unit tests (participle v2 migration)

Location: ~/projects/vfilter — branch participle-v2-upgrade (commits abfe2d4, 95a1f79; Inspect API uncommitted)

Command: go test ./... — all 8 packages pass:

Package	Result
vfilter	ok
arg_parser	ok
benchmarks	ok
explain	ok
marshal	ok
reformat	ok
scope	ok
utils	ok

Migration regression tests (pre-existing suite, all green after upgrade)

- TestVQLQueries / TestMultiVQLQueries — golden-file parse tests. Caught the comment-elision regression: v2's parser-level Elide no longer drops tokens at the lexer, so comments leaked into the AST. Fixed with a droppingLexerDef wrapper.
- TestMultiVQLQueries (multiline string) — caught missing (?s) flag on the MultilineString lexer rule.
- TestSerializaiton / TestVQLSerializaiton — caught position metadata polluting cmp.Diff after adding Pos/EndPos; fixed with compareOptionsWithPos ignoring lexer.Position.

Position capture verification (manual demo)

/tmp/opencode/posdemo/ — parses SELECT Foo, Bar(X=1) FROM Artifact.Linux.Sys() WHERE Foo > 3 and prints AST node spans. Verified every node maps exactly to its source offset:

Node	Expected	Actual
statement / SELECT	1:1-1:61	1:1-1:61
plugin Artifact.Linux.Sys	1:27-1:48	1:27-1:48
column Foo	1:8-1:11	1:8-1:11
column Bar(X=1)	1:13-1:22	1:13-1:22

2. vfilter Inspect() API tests

Location: ~/projects/vfilter/inspect_test.go — 2 tests, both pass

Test	What it verifies
TestInspect	Parses <code>LET Y = 5\nSELECT Foo(X=1), Bar FROM Artifact.Linux.Sys.Users() WHERE Foo > 3 AND baz(X=Y);</code> asserts 1 let (Y), 3 calls (plugin Artifact.Linux.Sys.Users with IsPlugin=true, Foo with arg X, baz), symbols include Y and Foo, and the plugin's Pos is at 2:27.
TestInspectNil	Inspect(nil) is nil-safe (no panic).

3. LSP diagnostic engine unit tests

Location: `velociraptor/vql/lsp/diagnostics_test.go` — 10 tests, all pass

Command: `go test ./vql/lsp/`

Test registry (fake): plugins `pslist` (arg `pid`), `Artifact.Linux.Sys.Users`; function `upcase` (arg `str`).

Test	Query	Expected diagnostics
TestValidateCleanDocument	<code>SELECT * FROM pslist(pid=1) WHERE Name =~ 'foo'</code>	none
TestValidateUnknownFunction	<code>SELECT upcase(str='x'), unknownfunc() FROM pslist()</code>	1: Unknown function 'unknownfunc' at 0:24
TestValidateUnknownPlugin	<code>SELECT * FROM bogusplugin()</code>	1: Unknown plugin 'bogusplugin' at 0:14
TestValidateUnknownArgument	<code>SELECT * FROM pslist(foo=1)</code>	1: Unknown argument 'foo' for plugin 'pslist' at 0:21
TestValidateMultilinePositions	bogus function on line 2	diagnostic at line 1 col 24 (0-based 1:24)
TestValidateSyntaxErrorOnly	<code>SELECT * FROM</code>	1 diag containing "unexpected token"
TestValidateTruncateAndRetry	4-line doc: bad arg on line 1, @@@ garbage on line 2, bad arg line 3, bogus plugin line 4	all 4: foo arg (0:21), lexer: invalid input text "@@" (1:7), bar arg (2:21), bogusplugin (3:14)
TestValidateTruncateAndRetrySyntaxErrorPosition	2 good lines then <code>SELECT @@@</code>	1 diag at line 2 col 7
TestValidateArtifactArgument	<code>Artifact.Windows.Sys.Users()</code> (via <code>AddArtifact</code> , arg <code>remoteRegKey</code>)	valid call clean; (foo=1) → Unknown argument 'foo' for artifact 'Artifact.Windows.Sys.Users' at Character 41
TestValidateKnownArtifactParameters	<code>Artifact.Generic.Client.VQL(Command='SELECT 5')</code> (arg <code>Command</code>)	clean

Key behaviors proven:

- Clean documents produce zero diagnostics.
- Unknown functions, plugins, and keyword arguments are each reported with exact byte-based ranges (0-based LSP positions).
- Syntax errors are reported with precise position from the underlying `*lexer.Error` / `participle.Error`.
- **Truncate-and-retry:** a syntax error mid-document does NOT suppress diagnostics from valid statements before or after it.

- Multi-line documents map positions to the correct line.

3b. LSP server lifecycle unit tests

Location: `velociraptor/vql/lsp/server_test.go` — **6 tests, all pass**

Uses a `mockClient` (embeds `protocol.UnimplementedClient`, records published diagnostics per URI) so no real stdio transport is needed.

Test	What it verifies
<code>TestServerInitializeCapabilities</code>	<code>initialize</code> returns full-sync <code>TextDocumentSync</code> , <code>DiagnosticProvider</code> (union — type-asserted to <code>*protocol.DiagnosticOptions</code>) with identifier <code>"vql"</code> , and <code>ServerInfo.Name == "vql-lsp"</code>
<code>TestServerDidOpenAndPullDiagnostic</code>	After <code>didOpen</code> of a bad doc, <code>pull Diagnostic()</code> returns the correct item AND push diagnostics fired
<code>TestServerDidChangeUpdatesDocument</code>	Full-sync <code>didChange</code> replaces the document; subsequent pull reports the new error
<code>TestServerDidCloseClearsDiagnostics</code>	<code>didClose</code> clears both the pull report and the pushed diagnostics
<code>TestServerPullUnknownDocument</code>	Pull on a never-opened URI returns empty items (no crash)
<code>TestServerShutdownClosesDone</code>	Shutdown closes the server's <code>Done()</code> channel

3c. Document symbol unit tests

Location: `velociraptor/vql/lsp/server_test.go` + `~/projects/vfilter/outline_test.go`

vfilter Outline() API — 2 tests, all pass

Test	What it verifies
<code>TestOutline</code>	<code>Outline()</code> produces a hierarchical outline of <code>LET Y = SELECT Foo FROM pslist(pid=1)\nSELECT upcase(str=X), Bar AS baz FROM Artifact.Linux.Sys.Users() WHERE Foo > 3: stmt1 root Y (let) → pslist (query) → Foo (column); stmt2 root Artifact.Linux.Sys.Users (query) → 2 columns (unaliased "" with upcase function child, aliased "baz")</code>
<code>TestOutlineNil</code>	<code>Outline(nil)</code> returns nil (no panic)

LSP server DocumentSymbol — 2 tests, all pass (total 18 in vql/lsp/)

Test	What it verifies
<code>TestServerDocumentSymbols</code>	After <code>didOpen</code> , <code>DocumentSymbol</code> returns the hierarchical outline: <code>Y (Variable) → pslist (Function) → Foo (Field); Artifact.Linux.Sys.Users (Function) → unaliased column (Field, name from source text) → upcase (Function), and baz (Field)</code>
<code>TestServerDocumentSymbolsUnknownDocument</code>	<code>DocumentSymbol</code> on a never-opened URI returns empty (no crash)

Kind mapping verified: `let`→`Variable(13)`, `query/function`→`Function(12)`, `column`→`Field(8)`. Initialize capabilities now advertise `DocumentSymbolProvider`:

true.

3d. Hover unit tests

Location: velociraptor/vql/lsp/server_test.go — 5 tests, all pass (total 23 in vql/lsp/)

Test	What it verifies
TestServerHoverPluginCall	Cursor on pslist in SELECT * FROM pslist(pid=1) → markdown contains pslist (plugin) and pid
TestServerHoverFunctionCall	Cursor on upcase → contains upcase (function) and str
TestServerHoverArgument	Cursor on pid arg → contains pid and int
TestServerHoverUnknownDocument	Never-opened URI → nil (no crash)
TestServerHoverUnknownSymbol	Cursor on a dynamic column (SomeDynamicColumn) → nil (no false positive)

Byte-offset gotcha: cursor positions are computed per document string (e.g. pslist is at byte 14 in SELECT * FROM pslist(pid=1)).

3e. Completion unit tests

Location: velociraptor/vql/lsp/server_test.go — 6 tests, all pass (total 29 in vql/lsp/)

Test	What it verifies
TestServerCompletionPluginPrefix	psl prefix → labels contain pslist
TestServerCompletionFunctionPrefix	upc prefix → labels contain upcase
TestServerCompletionArtifactPrefix	Art prefix → labels contain Artifact.Linux.Sys.Users
TestServerCompletionLetVariable	After LET X = 5\nSELECT * FROM → some item X with kind Variable
TestServerCompletionArguments	Inside pslist(→ some item pid with kind Field
TestServerCompletionUnknownDocument	Never-opened URI → empty labels

LET-variable recovery gotcha: when the document ends in an incomplete statement (whole-document parse fails), letNames falls back to largestParseablePrefix so LET variables are still offered.

Initialize capabilities advertise CompletionProvider with TriggerCharacters: [".", "("].

3f. Hybrid-mode artifact registry unit test

Location: velociraptor/vql/lsp/artifacts_test.go — 1 test, all pass (total 30 in vql/lsp/)

Test	What it verifies
TestBuildRegistryFromAPIClient	BuildRegistryFromAPIClient with a gomock MockAPIClient (api/mock) that returns two artifact descriptors (Windows.Sys.Users with param remoteRegKey, Generic.Client.VQL with param Command) — the registry resolves Artifact.Windows.Sys.Users (Doc "Enumerate users", Type "artifact", arg remoteRegKey), Artifact.Generic.Client.VQL (arg Command), and correctly reports Artifact.Bogus.Nope as absent

Uses `github.com/golang/mock/gomock` (already in `go.mod` as an indirect dep) and a real `vfilter.NewScope()`. This locks in the hybrid-mode registry builder: artifacts are fetched over the gRPC API rather than directly from the repository, so the exact plumbing the `lsp` command uses is covered.

4. End-to-end tests against the real binary

4a. `velociraptor lsp --check` (CLI smoke tests)

Binary: `~/local/bin/velociraptor` (also built ad-hoc to `/tmp/opencode/velociraptor-lsp` during development).

Query	Result
<code>SELECT * FROM pslist(pid=1) WHERE Name =~ 'foo'</code>	clean (no output)
<code>SELECT upcase(str='x'), * FROM pslist(foo=1) WHERE unknownfunc()</code>	3 diags: arg str on upcase (1:15), arg foo on pslist (1:39), unknown function (1:48)
<code>SELECT * FROM</code>	syntax diag: unexpected token "<EOF>" at 1:14
<code>SELECT * FROM pslist(badarg=1)</code>	Unknown argument 'badarg' for plugin 'pslist' at 1:22
<code>SELECT * FROM Artifact.Windows.Sys.Users()</code>	clean (artifact resolved from repository)
<code>SELECT * FROM Artifact.Windows.Sys.Users(foo=1)</code>	Unknown argument 'foo' for artifact ... at 1:42
<code>SELECT * FROM Artifact.Bogus.Nope()</code>	Unknown plugin 'Artifact.Bogus.Nope'
<code>SELECT * FROM Artifact.Generic.Client.VQL(Command='SELECT 5')</code>	clean
<code>Artifact.Generic.Client.VQL(Command='SELECT 5', badparam=1)</code>	Unknown argument 'badparam' for artifact ... at 1:63
<code>SELECT * FROM\ (EOF)</code>	diag at line 2 col 14, unexpected token <EOF>

4b. Full LSP protocol tests (Python stdio clients)

Python clients in `/tmp/opencode/` (`lsp_client.py`, `lsp_change.py`, `lsp_pull_test.py`, `kick_test.py`) speak Content-Length framed LSP over stdio.

Verified:

- **Handshake:** `initialize` returns capabilities `{textDocumentSync: 1, diagnosticProvider: {identifier: "vql", interFileDependencies: false, workspaceDiagnostics: false}}` and `serverInfo {name: "vql-lsp", version: "0.1"}`.
- **Document sync:** `didOpen/didChange` (full-sync) update the document; `didClose` clears diagnostics.
- **Push diagnostics:** `textDocument/publishDiagnostics` fires on open/change with the correct diagnostic list.
- **Pull diagnostics:** `textDocument/diagnostic` returns a `RelatedFullDocumentDiagnosticReport` with `kind: "full"` and the same diagnostics (LSP 3.17 style, used by `opencode`).
- **Lifecycle:** `shutdown` → `{"result": null}`, `exit` → process exits 0.
- A bad doc (`Artifact.Windows.Sys.Users(foo=1)`) returns Unknown argument 'foo' for artifact 'Artifact.Windows.Sys.Users' at 0:41 through BOTH push and pull channels.

- **Document symbols:** `textDocument/documentSymbol` returns the hierarchical outline for a two-statement doc (`LET Y = SELECT Foo FROM pslist(pid=1)\nSELECT upcase(str=X), Bar AS baz FROM Artifact.Linux.Sys.Users() WHERE Foo > 3): Y (Variable) → pslist (Function) → Foo (Field); Artifact.Linux.Sys.Users (Function) → upcase(str=X) (Field, name from source) → upcase (Function), and baz (Field).` (Python client `/tmp/opencode/lsp_symbols_test.py`; note the push notification arrives before the response — drain loop required.)
- **Hover:** `textDocument/hover` on `upcase` in `SELECT upcase(str='x') FROM pslist(pid=1)` returns markdown with the REAL registry doc — `**upcase** (function) + "Returns the uppercase version of a string." + argument string (real name, not str)`; hover on the `pid` argument returns `**pid** – int64 (real type)`. Capabilities advertise `hoverProvider: true`. (Python client `/tmp/opencode/lsp_hover_test.py`.)
- **Completion:** `textDocument/completion` with prefix `psl → pslist`; `upc → upcase`; `Art → full artifact tree (Artifact.ADX.Flows.Upload, Artifact.Admin.Client.*, ...)`; `LET variable X after SELECT * FROM`; argument `pid` inside `pslist()`. Capabilities advertise `completionProvider` with trigger characters `.` and `(`. (Python client `/tmp/opencode/lsp_completion_test.py`; ALL OK — see the test file's own output for the 5-case table.)

4c. Hybrid-mode e2e tests (--datastore + local API server)

Binary built ad-hoc to `/tmp/opencode/velociraptor-hybrid` during development (`go build -o /tmp/opencode/velociraptor-hybrid ./bin/`).

Scenario	Command / setup	Result
Default port free	<code>lsp --check "SELECT * FROM Artifact.Windows.Sys.Users()" on a fresh datastore</code>	clean, exit 0 (~2.3s: config gen + local API start)
Error diagnostics	<code>--check "SELECT upcase(str='x'), bogusfunc() FROM pslist(badarg=1)"</code>	3 diags (1:15 str/upcase, 1:25 bogusfunc, 1:49 badarg/pslist)
Custom artifact resolves	placed <code>Custom/Test.yaml</code> (name <code>Custom.Test</code> , param <code>flavor</code>) in <code><datastore>/artifact_definitions/</code> ; <code>--check "SELECT * FROM Artifact.Custom.Test(flavor='chocolate')"</code>	clean, exit 0 — custom artifacts visible through hybrid mode
Custom artifact unknown param	<code>--check "SELECT * FROM Artifact.Custom.Test(badparam=1)"</code>	Unknown argument 'badparam' for artifact 'Artifact.Custom.Test' at 1:36
Custom artifact missing	<code>--check "SELECT * FROM Artifact.Bogus.Nope()"</code>	Unknown plugin 'Artifact.Bogus.Nope' at 1:15
Port collision avoided	a dummy process held <code>127.0.0.1:8001</code> ; ran the same custom-artifact check	clean, exit 0 — LSP silently picked a free ephemeral port

Datastore layout fact: with the generated config (`FilestoreDirectory = datastore_directory`), custom artifacts live at `<datastore>/artifact_definitions/<OrgPath>/<Artifact>.yaml` (the same layout paths.`GetArtifactDefintionPath` produces).

5. opencode integration test (the "kick the tires" test)

- Config: `/home/me/.config/opencode/opencode.json` — custom LSP server `{"command": ["velociraptor", "lsp"], "extensions": [".vql"]}`.
- Binary deployed to `~/.local/bin/velociraptor` (on PATH).
- After restarting opencode and opening `lsp-test/test.vql`, the server process was observed running (velociraptor lsp, PID observed, CPU settling after startup).
- Test file `lsp-test/broken.vql` containing five deliberate errors:

```
SELECT upcase(str='x'), bogusfunc() FROM Artifact.Windows.Sys.Users(foo=1)
SELECT * FROM Artifact.Bogus.Nope() SELECT @@
```

Writing the file produced the following diagnostics injected directly into the agent's tool result (opencode agent feedback loop):

```
ERROR [3:8] lexer: invalid input text "@@\n" ERROR [1:15] Unknown argument
'str' for function 'upcase' ERROR [1:25] Unknown function 'bogusfunc' ERROR
[1:69] Unknown argument 'foo' for artifact 'Artifact.Windows.Sys.Users'
ERROR [2:15] Unknown plugin 'Artifact.Bogus.Nope'
```

All five error classes detected, including diagnostics both before AND after the line-3 syntax error (truncate-and-retry working in the real client). The str diagnostic was cross-checked against velociraptor vql list: upcase genuinely takes string, not str — confirmed a true positive.

6. Agent-discovery option tests (see VQL-LSP-USAGE.md)

6a. validate-vql custom opencode tool (Option 2)

The custom tool at `~/.config/opencode/tools/validate-vql.ts` shells out to `velociraptor lsp --check` and parses the output into structured JSON. After an opencode restart, the tool appears in the agent's function list. Verified from inside the live agent:

Query	Result
<code>SELECT upcase(str='x'), bogusfunc() FROM Artifact.Windows.Sys.Users(foo=1)</code>	<code>valid: false, 3 diagnostics (1:15 str/ upcase, 1:25 bogusfunc, 1:69 foo/artifact)</code>
<code>SELECT * FROM Artifact.Windows.Sys.Users()</code>	<code>valid: true, diagnostics []</code>
<code>SELECT * FROM pslist(pid=1)</code>	<code>valid: true, diagnostics []</code>

Also `bun build tools/validate-vql.ts` compiles cleanly (74 modules, 0.42 MB).

6b. velociraptor-vql-validation skill (Option 1)

Skill file created at `~/.config/opencode/skills/velociraptor-vql-validation/SKILL.md`; its quick-start command (`velociraptor lsp --check`) is the same path proven in sections 4 and 6a. Skill frontmatter auto-matches when a task mentions writing/checking/debugging VQL.

6c. mcpls LSP→MCP bridge (Option 3a)

mcpls v0.3.8 (rebuilt from source — prebuilt binary required glibc 2.39, system has 2.35) wraps the vql LSP server and exposes MCP tools. Verified with a newline-delimited-JSON MCP client:

- initialize → `protocolVersion 2024-11-05, serverInfo {name mcpls, title "MCPLS - MCP`

- to LSP Bridge", version 0.3.8}.
- tools/list → 20 tools including get_diagnostics.
- tools/call get_diagnostics on /home/me/projects/velociraptor/lsp-test/bridge-test.vql (2-line bad file) returned both diagnostics as JSON: Unknown argument 'foo' for artifact 'Artifact.Windows.Sys.Users' (1:42) and Unknown argument 'badarg' for plugin 'pslist' (2:22).

Config in /home/me/.config/mcpls/mcpls.toml (roots restricted to /home/me/projects/velociraptor; [[workspace.language_extensions]] for vql; [[lsp_servers]] entry running velociraptor lsp with handles = ["diagnostics"]).

6d. Native MCP tool validate_vql in mcp-velociraptor (Option 3b)

Tool implemented in /home/me/projects/mcp-velociraptor at internal/tools/validate_vql.go (lazy cached registry via lsp.BuildRegistryWithArtifacts; no API connection required for validation itself, though the server requires an API config at startup). Verified with a newline-delimited-JSON MCP client both standalone and against a **live Velociraptor instance** (API config at /velociraptor/datastore5/api.config.yaml, localhost:8001):

- tools/list → [list_clients, validate_vql].
- validate_vql bad query → valid: false, 3 diagnostics (1:15, 1:25, 1:69) exactly matching the LSP/CLI/tool results.
- validate_vql good query → valid: true, diagnostics [].
- list_clients against the live instance returned 3 real clients (hostname localhost, linux linuxmint21.3), proving the shared registry and API connection work together.

Summary of coverage

Error class	Unit test	CLI test	LSP test	opencode
Syntax error	✓	✓	✓	✓
Unknown function	✓	✓	✓	✓
Unknown plugin	✓	✓	✓	✓
Unknown keyword arg (plugin)	✓	✓	✓	—
Unknown keyword arg (artifact)	✓	✓	✓	✓
Unknown artifact	✓	✓	—	✓
Multi-line position accuracy	✓	—	—	✓
Truncate-and-retry (errors after syntax error)	✓	✓	—	✓
Clean document → no diagnostics	✓	✓	✓	✓
Pull-based diagnostics	—	✓	✓	✓
Push diagnostics on open/change	—	✓	✓	—
didClose clears diagnostics	—	✓	✓	—
Shutdown/exit lifecycle	—	✓	✓	—
Initialize capabilities (full sync, provider, serverInfo)	—	✓	✓	✓
Document symbols (hierarchical outline, kinds)	—	✓	✓	—
Hover (function/plugin/argument docs + types)	—	✓	✓	—
Completion (plugins, functions, artifacts, LET vars, args)	—	✓	✓	—
Hybrid mode: custom artifacts in datastore resolve	✓	✓	—	—
Hybrid mode: unknown param on custom artifact	✓	✓	—	—
Hybrid mode: automatic port-collision avoidance	—	✓	—	—

Artifact registry facts discovered during testing

- The base scope (from `vql_subsystem.MakeScope()`) contains ~437 built-in plugins/functions but NOT artifacts.
- The repository stores artifacts WITHOUT the `Artifact.` prefix (e.g. `Windows.Sys.Users`); VQL references them WITH the prefix. The LSP registry prepends `Artifact.` when registering.
- Real artifact parameter examples: `Generic.Client.Info` → 0 params, `Windows.Sys.Users` → 1 (`remoteRegKey`), `Generic.Client.VQL` → 1 (`Command`).
- Hybrid mode fetches artifacts over the API (`GetArtifacts`); custom artifacts on disk are stored under `<datastore>/artifact_definitions/`.
- A real-world port conflict was observed during development: a running `velociraptor` gui on `127.0.0.1:8001` caused `lsp` to fail with `bind: address already in use` — this motivated the automatic port-collision avoidance (the command now picks a free ephemeral port instead of failing).